

Q1 [GATE CSE 2004]: An input sequence of elements 1, 2, 3, 4, 5 arrives in strict sequential order. Which of the following permutations cannot be obtained as an output sequence using a single stack?

Answer: 3, 1, 2, 5, 4

Explanation: To get 3 as the first output, 1, 2, and 3 must be pushed in sequence. The stack contents are then [1, 2, 3] with 3 at the top. Once 3 is popped, the top element becomes 2. Therefore, it is structurally impossible to pop 1 before popping 2. Thus, the sequence 3, 1 cannot be achieved.

Q2 [GATE CSE 2013]: What is the worst-case time complexity of executing a sequence of n push and pop operations on an initially empty stack implemented using a dynamically resizing array that doubles its capacity whenever it becomes full?

Answer: $O(n)$

Explanation: While an individual push triggering array reallocation takes linear time to copy existing elements, doubling happens infrequently at powers of 2. Summing up all copy steps yields a geometric series: $1 + 2 + 4 + \dots + n \leq 2n$ operations. Through amortized analysis, the average cost per operation is $O(1)$, making the total worst-case time complexity for n operations $O(n)$.

Q3 [GATE CSE 2015]: Assume operators $+$, $-$, $*$ are left-associative, while exponentiation $\wedge$ is right-associative. What is the correct postfix equivalent of the infix expression: $a + b * c - d \wedge e \wedge f$?

Answer: $a\ b\ c\ * +\ d\ e\ f\ \wedge\ \wedge -$

Explanation: Due to right-associativity, $d \wedge e \wedge f$ is parsed from right to left as $(d \wedge (e \wedge f))$, converting to postfix as $def^{\wedge\wedge}$. Multiplication has higher precedence than addition, so $b * c$ becomes bc^* . The expression reduces to $a + bc^* - def^{\wedge\wedge}$. Left-to-right evaluation of equal-precedence $+$ and $-$ operations yields abc^*+ followed by the subtraction, resulting in $abc^*+def^{\wedge\wedge}-$.

Q4 [GATE CSE 2007]: What is the minimum number of stacks required to efficiently simulate a First-In-First-Out (FIFO) Queue data structure?

Answer: 2 Stacks

Explanation: A single stack reverses the arrival sequence due to its LIFO nature. By popping items out of the first stack and pushing them into a second stack, the original FIFO order is fully restored.

Q5 [GATE CSE 1998]: The postfix expression $6\ 2\ 3\ +\ *\ 3\ 8\ 2\ /\ /\ +$ is evaluated using a stack. What is the maximum number of elements present in the stack at any point during evaluation?

Answer: 4 elements

Explanation: Operands are pushed onto the stack sequentially. The maximum stack depth occurs when evaluating the sub-expression values. Pushing 6, then 2, then 3 gives a depth of 3. After evaluation, operations continue until a sequence of operands like 3, 8, 2 is accumulated over previous results, hitting a maximum of 4 nodes simultaneously on the stack.

Q6 [GATE CSE 1996]: An empty stack processes characters arriving in sequence A, B, C, D, E. An interleaved sequence of push and pop operations outputs characters. Which of the following output strings can be legally generated?

Answer: B, C, D, E, A

Explanation: This is generated via: push(A), push(B), pop() $\rightarrow$ B, push(C), pop() $\rightarrow$ C, push(D), pop() $\rightarrow$ D, push(E), pop() $\rightarrow$ E, pop() $\rightarrow$ A. All other options violate LIFO rules by attempting to pop deeply nested elements early.

Q7 [GATE CSE 1991]: Which data structure is explicitly required by compilers to manage activation records and local variable allocations for block-scoped recursive functions?

Answer: Stack

Explanation: Function calls execute in nested scopes. A stack naturally tracks runtime execution contexts because a function cannot exit until all internal nested calls finish executing, matching LIFO semantics.

Q8 [GATE CSE 2019]: Let a stack be implemented using an array. If the initial top pointer is set to -1, what are the sequence of pointer states during a push followed immediately by a pop operation?

Answer: Top changes from -1 to 0, then returns to -1.

Explanation: Pushing increments the index before inserting data (top++), shifting the index from -1 to 0. Popping reads the data and decrements the tracking variable (top--), returning the value back to -1.

Q9 [GATE CSE 2002]: What is the postfix form of the infix expression $A + B * (C + D)$?

Answer: A B C D + * +

Explanation: Parentheses dictate immediate grouping: $(C + D) \rightarrow CD+$. The expression becomes $A + B * CD+$. Multiplication takes precedence over addition: $B * CD+ \rightarrow BCD*+$. Finally, the remaining addition is resolved, yielding $ABCD+*+$.

Q10 [GATE CSE 2016]: A single stack is used to reverse the words of a string in place. If the string contains k words each of average length m , what is the best-case time complexity of the reversal algorithm?

Answer: $O(k \times m)$ which is linear with respect to string length.

Explanation: Each character of each word must be pushed once into the stack and popped once to reverse that word. The number of total characters is proportional to $k \times m$, resulting in strict linear time.

Q11 [GATE CSE 1995]: If a stack is allocated a fixed continuous memory space of size M , what is the condition to identify a Stack Overflow state?

Answer: $\text{top} == M - 1$ (for a 0-indexed array)

Explanation: When the top index reaches the last slot of the array, any additional push operations violate bounded memory constraints, triggering an overflow error.

Q12 [GATE CSE 2021]: Consider an empty stack. Execute operations: push(54); push(52); pop(); push(55); push(62); s = pop(). What is the value of variable s?

Answer: 62

Explanation: Tracing: push(54) $\rightarrow$ [54]; push(52) $\rightarrow$ [54, 52]; pop() $\rightarrow$ removes 52; push(55) $\rightarrow$ [54, 55]; push(62) $\rightarrow$ [54, 55, 62]. The final pop() extracts the top element, which is 62.

Q13 [GATE CSE 2014]: Which properties characterize a stack's internal storage behavior?

Answer: Linear, sequential access restricted to one end.

Explanation: A stack organizes its elements in a linear sequence. Unlike arrays, random access is forbidden; data can only be modified or read from the top pointer.

Q14 [GATE CSE 1993]: What is the postfix evaluation of 5 3 2 * +?

Answer: 11

Explanation: Push 5, then 3, then 2. Encounter *: pop 2 and 3, multiply ($3 \times 2 = 6$), push 6. Stack contains [5, 6]. Encounter +: pop 6 and 5, add ($5 + 6 = 11$), push 11.

Q15 [GATE CSE 2003]: What is the prefix notation for the infix expression $(A + B) * C$?

Answer: $* + A B C$

Explanation: Convert parenthesized terms first: $(A + B) \rightarrow +AB$. The expression becomes $(+AB) * C$. Converting the multiplication operation shifts the operator to the front, yielding $* + A B C$.

Q16 [GATE CSE 2008]: Can a standard stack be utilized to check if a string is a palindrome?

Answer: Yes

Explanation: Push the first half of the string's characters onto the stack, then compare the remaining characters with the values popped from the stack one by one. If all characters match, the string is a palindrome.

Q17 [GATE CSE 2011]: What happens when a function call is executed in a recursive program?

Answer: A new activation record is pushed onto the call stack.

Explanation: The activation record stores local copies of parameters, variables, and the return address. This guarantees that nested scopes can execute without modifying parameters in parent scopes.

Q18 [GATE CSE 2012]: A circular queue of capacity $n - 1$ elements is implemented using an array of size n . REAR and FRONT are array index tracking variables. Initially $REAR = FRONT = 0$. What are the conditions used to detect when the queue is FULL and when it is EMPTY?

Answer: Empty: $REAR == FRONT$; Full: $(REAR + 1) \% n == FRONT$

Explanation: When $REAR == FRONT$, the queue has no elements. To differentiate between a completely full queue and an empty queue, one array slot is intentionally left empty. The queue is considered full when incrementing REAR points directly to FRONT.

Q19 [GATE CSE 2014]: If we restrict insertions and deletions in a double-ended queue (Deque) such that elements can only be inserted at the rear but can be removed from both the front and the rear, what structural variant is created?

Answer: Input-Restricted Deque

Explanation: Deques allow data access at both boundaries. Restricting insertions to a single end while keeping both ends open for removals creates an Input-Restricted Deque.

Q20 [GATE CSE 2016]: A priority queue is implemented using an initially empty Min-Heap. If n items are inserted sequentially, what is the total worst-case time complexity to extract all items one by one in sorted order?

Answer: $O(n \log n)$

Explanation: Inserting n items takes $O(n \log n)$ time. Extracting the minimum element requires rebuilding the heap, which takes $O(\log n)$ time per extraction. Performing this operation n times takes $O(n \log n)$ time, yielding an overall complexity of $O(n \log n)$.

Q21 [GATE CSE 2004]: How many stacks are required to implement a FIFO queue when no other storage formats or memory collections are available?

Answer: 2 Stacks

Explanation: Two stacks are required: one to handle incoming items, and a second to hold reversed items for dequeue operations, ensuring FIFO behavior.

Q22 [GATE CSE 1999]: In an array-based implementation of a simple linear queue, what is the major limitation encountered after multiple enqueue and dequeue cycles?

Answer: False Overflow

Explanation: Dequeue operations leave empty slots at the front of the array. However, the rear pointer continues moving forward linearly. Once rear reaches the last index, the queue rejects new insertions even if there is available space at the front.

Q23 [GATE CSE 2021]: Consider an empty queue. Execute operations: enqueue(21); enqueue(24); dequeue(); enqueue(28); enqueue(32); q = dequeue(). What is the value of variable q?

Answer: 24

Explanation: Tracing: enqueue(21)→[21]; enqueue(24)→[21, 24]; dequeue() removes 21, leaving [24]; enqueue(28)→[24, 28]; enqueue(32)→[24, 28, 32]. The next dequeue() removes the oldest remaining item, which is 24.

Q24 [GATE CSE 1995]: What type of queue scheduling algorithm is commonly used in CPU round-robin thread allocation models?

Answer: FIFO Circular Queue

Explanation: Round-robin scheduling gives each task a fixed time slice. Active tasks are placed in a queue; when a task's time slice expires, it is moved to the back of the queue, ensuring fair and orderly execution.

Q25 [GATE CSE 2015]: What is the optimal time complexity to look up the maximum value in a Monotonic Decreasing Queue?

Answer: $O(1)$

Explanation: A monotonic decreasing queue keeps elements in strictly descending order. This guarantees that the largest element always sits at the front of the queue, allowing it to be retrieved in constant time.

Q26 [GATE CSE 2005]: Which graph traversal algorithm uses a queue as its primary tracking data structure?

Answer: Breadth-First Search (BFS)

Explanation: BFS explores nodes layer by layer based on their proximity to the starting node. A queue ensures that neighboring nodes are processed in the exact order they were discovered.

Q27 [GATE CSE 2017]: A queue is implemented using two stacks, S_1 and S_2 . Enqueue pushes directly to S_1 . Dequeue pops from S_2 , or transfers all elements from S_1 to S_2 if S_2 is empty. What is the amortized cost of a dequeue operation?

Answer: $O(1)$

Explanation: Although transferring elements takes linear time, each element is moved between the stacks at most once. The total cost across a sequence of operations averages out to a constant cost per operation.

Q28 [GATE CSE 1997]: What is the primary difference between a standard queue and a priority queue?

Answer: Retrieval order criteria.

Explanation: Standard queues process elements strictly in order of arrival (FIFO). Priority queues retrieve elements based on an assigned priority value, regardless of when they entered the queue.

Q29 [GATE CSE 2001]: What is the mathematical condition for an underflow state in an array-based circular queue?

Answer: $\text{front} == \text{rear}$

Explanation: In a circular queue, when the front and rear pointers converge to the same index, it indicates that all elements have been removed and the queue is completely empty.

Q30 [GATE CSE 2018]: Can a queue structure be used to implement an asynchronous communication buffer between two processes running at different speeds?

Answer: Yes

Explanation: A queue acts as a temporary buffer. The faster process appends data to the rear, while the slower process reads data from the front, decoupling their execution speeds.

Q31 [GATE CSE 1994]: If a queue is implemented using a circular linked list, which pointer configuration allows for constant-time ($O(1)$) enqueue and dequeue operations?

Answer: Maintaining a single pointer to the tail node.

Explanation: In a circular linked list, the tail node points directly back to the head node ($\text{tail->next} == \text{head}$). This gives you instant access to both the front and the back of the queue using a single pointer.

Q32 [GATE CSE 2009]: What is the space complexity of a breadth-first search traversal on a graph with maximum breadth B ?

Answer: $O(B)$

Explanation: The queue must store all nodes at the widest layer of the graph simultaneously, making the space complexity proportional to the graph's maximum breadth.

Q33 [GATE CSE 2020]: Which data structure is used to implement a network packet buffer that processes data packets in their exact order of arrival?

Answer: FIFO Queue

Explanation: Network buffers must preserve packet ordering to ensure data is reassembled correctly, making a first-in, first-out queue the correct choice.

Section 3: Linked List Data Structure (Questions 34 to 50)

Q34 [GATE CSE 2010]: The following C function takes a singly-linked list as input, moves the last element to the front of the list, and returns the updated head pointer. What code correctly fills the blank line?

```
Node *move_to_front(Node *head) {  
    Node *p, *q;  
    if ((head == NULL) || (head->next == NULL)) return head;  
    q = NULL; p = head;
```

```
while (p->next != NULL) { q = p; p = p->next; }  
// LINE BLANK
```

Q35 [GATE CSE 2016]: A queue is implemented using a non-circular singly linked list with a head and a tail pointer. Enqueue inserts a new node at the head, and dequeue removes a node from the tail. What is the worst-case time complexity of these operations?

Answer: Enqueue: $O(1)$; Dequeue: $O(n)$

Explanation: Inserting at the head takes constant time ($O(1)$). However, removing from the tail requires updating the tail pointer to point to its predecessor node. Because this is a singly linked list, finding the predecessor requires traversing the entire list from the head, taking linear time ($O(n)$).

Q36 [GATE CSE 2012]: A singly linked list contains a loop. A fast pointer (moving 2 nodes per step) and a slow pointer (moving 1 node per step) start together from the head node. If the distance from the head to the start of the loop is X nodes, and the loop contains Y nodes, after how many iterations will they meet?

Answer: The first integer multiple of Y that is greater than or equal to X .

Explanation: The fast pointer travels at twice the speed of the slow pointer. For them to meet inside the loop, the difference in the total distance they traveled must be an integer multiple of the loop's length (Y), and they must have traveled far enough to enter the loop ($k \geq X$).

Q37 [GATE CSE 2011]: You are given a pointer p to an intermediate node inside a singly linked list. You do not have access to the head pointer. What is the time complexity to delete node p ?

Answer: $O(1)$

Explanation: Since you cannot find the predecessor node directly, you can copy the data from the next node into node p , and then delete the next node instead. This achieves the same result in constant time.

Q38 [GATE CSE 2005]: What is the minimum time complexity and extra space required to reverse a singly linked list of n elements iteratively?

Answer: Time Complexity: $O(n)$; Space Complexity: $O(1)$

Explanation: A linked list can be reversed in a single pass by updating each node's next pointer in place using three temporary pointer variables (prev, curr, and next), requiring constant extra space.

Q39 [GATE CSE 2018]: Which of the following statements is true regarding arrays and linked lists?

Answer: Arrays provide $O(1)$ random access but have higher insertion costs due to element shifting; linked lists require linear traversal for lookups.

Explanation: Arrays store elements in contiguous memory, allowing direct index lookups. Linked lists use pointers scattered across memory, requiring a sequential scan to find elements but allowing fast insertions once the position is found.

Q40 [GATE CSE 2003]: What is the worst-case time complexity to sort a singly linked list of n elements using the Merge Sort algorithm?

Answer: $O(n \log n)$

Explanation: Merge Sort splits the list in half recursively ($O(\log n)$ levels) and merges the sub-lists back together ($O(n)$ work per level). This holds true for linked lists and does not require random access.

Q41 [GATE CSE 1992]: What is the memory overhead associated with a Doubly Linked List compared to a Singly Linked List?

Answer: One extra pointer field per node.

Explanation: Each node in a singly linked list stores one pointer (next). A doubly linked list node must store two pointers (next and prev), doubling the memory overhead used for pointer tracking.

Q42 [GATE CSE 2006]: Suppose a singly linked list is used to implement a stack. Where should the push and pop

operations occur to ensure optimal performance?

Answer: At the head node.

Explanation: Inserting or deleting a node at the head of a linked list takes constant time ($O(1)$) and does not require traversing the list, matching a stack's performance requirements.

Q43 [GATE CSE 1995]: What is the primary cause of a memory leak when working with dynamically allocated linked lists?

Answer: Losing the reference pointer to a node without freeing its allocated memory.

Explanation: If you overwrite a node's pointer without explicitly calling `free()` or `delete`, that block of memory remains allocated in the heap but becomes unreachable by the program.

Q44 [GATE CSE 2013]: What is the purpose of using a dummy header node in a linked list implementation?

Answer: To simplify code logic by eliminating edge cases for empty lists.

Explanation: A dummy node ensures that every valid data node always has a preceding neighbor, removing the need for special checks when inserting or deleting at the head of the list.

Q45 [GATE CSE 2001]: What is the worst-case time complexity to concatenate two singly linked lists of lengths m and n if you only have pointers to their head nodes?

Answer: $O(m)$

Explanation: To link the two lists, you must traverse the first list from its head to find its terminal tail node, which takes time proportional to the first list's length (m).

Q46 [GATE CSE 2008]: Can a circular linked list be used to implement a queue efficiently without maintaining a head pointer?

Answer: Yes, by maintaining a pointer to the tail node.

Explanation: In a circular linked list, the tail node points directly to the head node (`tail->next == head`). This gives you instant access to both ends of the queue using a single pointer reference.

Q47 [GATE CSE 2017]: Which data structure combination is ideal for implementing an LRU (Least Recently Used) cache?

Answer: A Hash Map and a Doubly Linked List.

Explanation: The doubly linked list allows cache pages to be moved or removed in constant time ($O(1)$) to track usage recency, while the hash map provides fast lookups ($O(1)$) to check if a page is already in the cache.

Q48 [GATE CSE 1996]: What is a major disadvantage of linked lists compared to arrays regarding hardware cache optimization?

Answer: Poor CPU cache locality.

Explanation: Arrays store elements in contiguous blocks of memory, which optimizes hardware pre-fetching. Linked list nodes are scattered across the heap, leading to frequent CPU cache misses during traversal.

Q49 [GATE CSE 2019]: What is the time complexity to find the middle node of a linked list using the two-pointer method?

Answer: $O(n)$

Explanation: While the fast pointer allows you to locate the middle node in a single pass, you still need to traverse through all n nodes in the list, resulting in a linear time complexity.

Q50 [GATE CSE 2022]: How does a Skip List achieve logarithmic search times?

Answer: By building an express-lane hierarchy of pointers over a standard linked list.

Explanation: Skip lists use multiple layers of forward pointers that skip over blocks of nodes. This allows search queries to bypass large sections of data, mimicking a binary search over a linked list and reducing search times to $O(\log n)$.